Interrupts

ECE 2564, Embedded Systems

Fall 2025

Overview

- What are interrupts?
- Why do we need interrupts?
- How are interrupts implemented?

Motivation

- Polling versus interrupts
  - Do you check your phone every minute? – Polling
  - Suppose you check your phone only when you receive a notification – Interrupt-driven
- By default, a CPU needs to poll its peripherals to find out when they need attention
  - Example: a timer rolls over
  - Example: an input buffer is full
  - Example: input pin X on a GPIO port has changed
- Polling can be wasteful, especially if a peripheral does not need attention very often

Interrupts

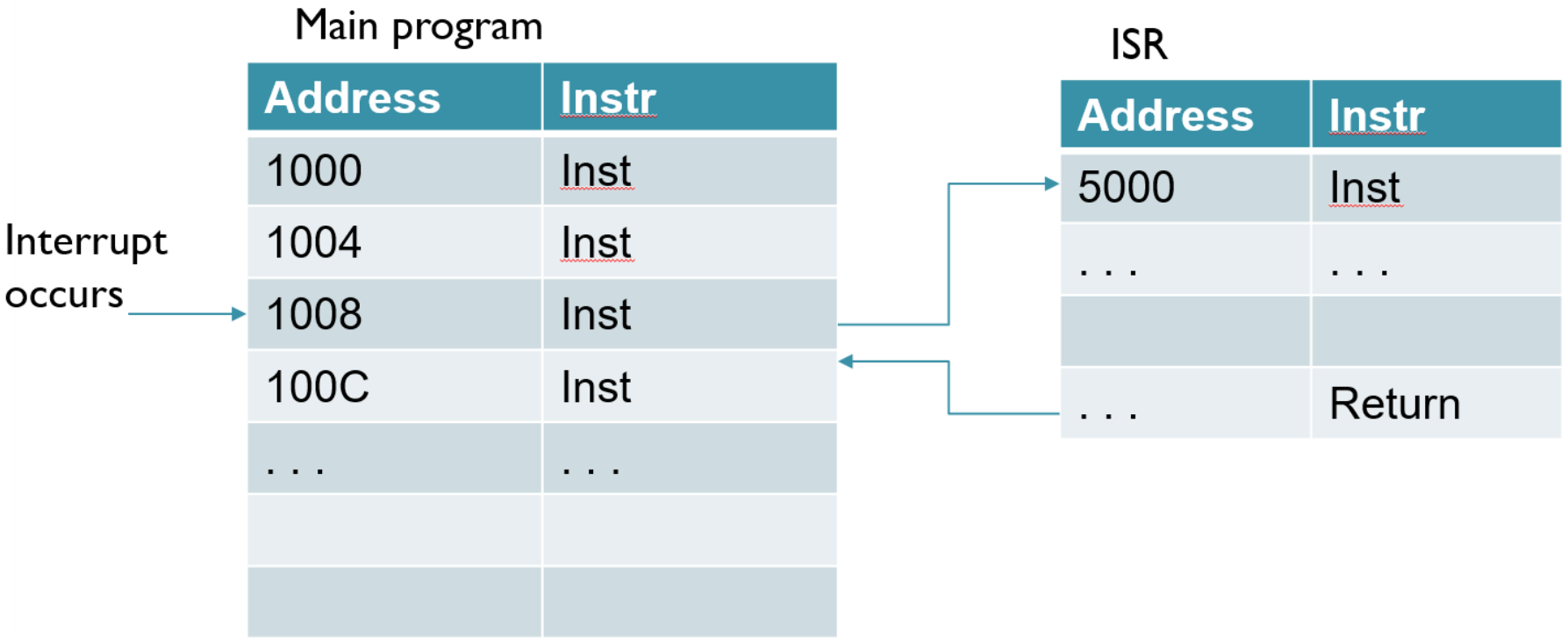
- Most processors allow peripherals to *interrupt* the normal sequence of instruction execution

- Example: a timer rolls over
  - The timer hardware generates an "interrupt request"
- Example: a UART receives a character
  - The UART hardware generates an "interrupt request"

- In response to an interrupt request (IRQ), the processor executes an Interrupt Service Routine (ISR), also known as an Interrupt Handler

- The main program is unaware that an ISR has executed

Basic ISR Idea



Basic idea behind an ISR

Interrupt Action Sequence

- A device asserts an Interrupt Request
- The processor completes the current instruction
- The processor saves its current state, possibly by writing to memory the contents of certain registers *including the current value in the Program Counter*
- The address of the first instruction in the Interrupt Service Routine is placed in the Program Counter
- The processor executes instructions in the ISR
- To return from the interrupt, the processor restores the state (registers & flags) that it saved at the beginning of this sequence
- When returning from the interrupt, *the Program Counter value that was stored previously* is loaded into the Program Counter
- The processor resumes execution from its previous spot in the main program

Processor Operation: What Happens Normally?

- REPEAT
  - FETCH the current instruction
  - ADVANCE the Program Counter (PC)
  - DECODE the just-fetched instruction
  - EXECUTE the just-decoded instruction

Processor Operation: What Happens When Executing an Interrupt?

- The usual sequence can be altered:
  - *If interrupts have been enabled*, and *if a peripheral has raised an interrupt request*, then the CPU's hardware takes special actions that involve saving and then changing the PC
  - These actions prepare the CPU to execute the ISR

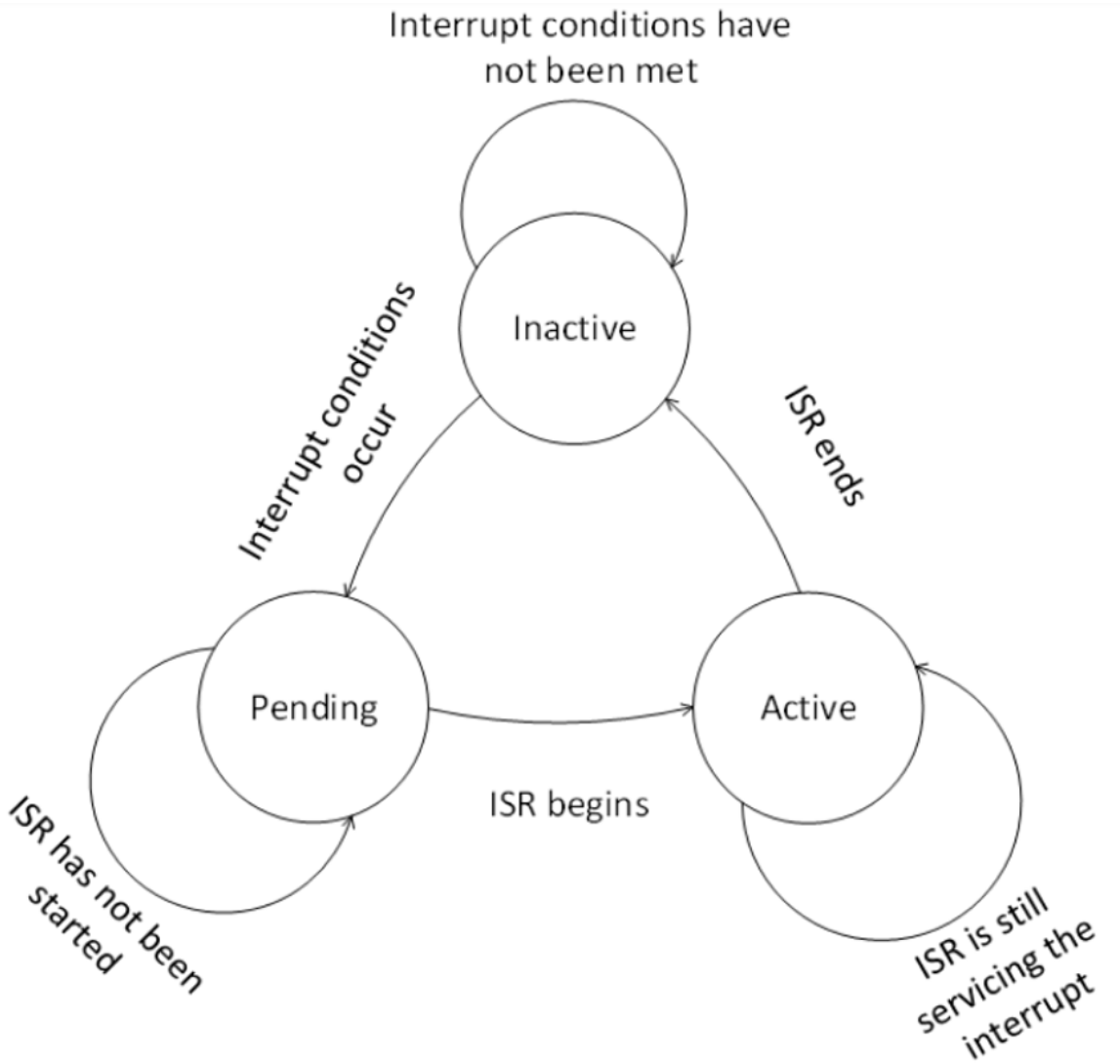
Interrupts

- An interrupt occurs when a device generates a signal (maybe on a physical pin) to get the processor's attention
- The processor suspends the current program and jumps to an Interrupt Service Routine (ISR)
- When the ISR finishes, the processor returns to the current program
- Servicing an interrupt combines hardware and software aspects of the processor
  - It is very much like a hardware-initiated call to a function (except that no parameters are passed, and there is no return value)

Some Potential Complications

- Important: The occurrence of an interrupt can be unrelated to the process that is currently executing
- Interrupts are said to be *asynchronous*
  - They can be generated by devices at any time
- *Nested* interrupts are possible
  - An interrupt can occur while an ISR is already executing

Modeling the Handling of an Interrupt With a FSM



- Any interupt will go through this cycle of states
- Sources of interrupts:
  - UART, GPIO, ADC, software, ...

Interrupt Service Routines (aka Interrupt Handlers)

- An ISR resembles a C function but has the following key differences:
  - It is *not* called by software (e.g., you don't call the from `main`)
  - An ISR's first instruction must be located at a known memory address
    - This is often reflected in the fact that they must have very specific names
  - Other functions cannot pass arguments to the ISR
  - The ISR cannot return values

- If the ISR isn't called by software, how does it execute? And if it can't receive or return values, how does it interact with your program?

Servicing Interrupts

- The location of the ISR depends on the design of the microprocessor
- Two common approaches involve the use of *fixed interrupts* and *vectored interrupts*
- Many desktop processors support both approaches

Handling Interrupts: Another Perspective

- The processor is executing instructions in some function F, when a peripheral raises an interrupt request
- The processor completes the current machine instruction and saves the current *context* — this refers to the current CPU register values in general, and the PC value in particular
- The *address* of the relevant ISR (the *interrupt vector*) is loaded into the program counter (PC)

- The processor executes the ISR
- The ISR is like a function; its purpose is to take care of the peripheral device that generated the interrupt

---

- When the ISR exits, the saved processor context is reloaded
- Re-loading the (previously saved) PC value causes the processor to resume normal execution in function F, at the next machine instruction

Fixed Interrupts

- For a fixed interrupt, the interrupt vector is built into the microprocessor and *cannot* be changed
- Generally, the user must ensure that the ISR is stored in memory starting at that address

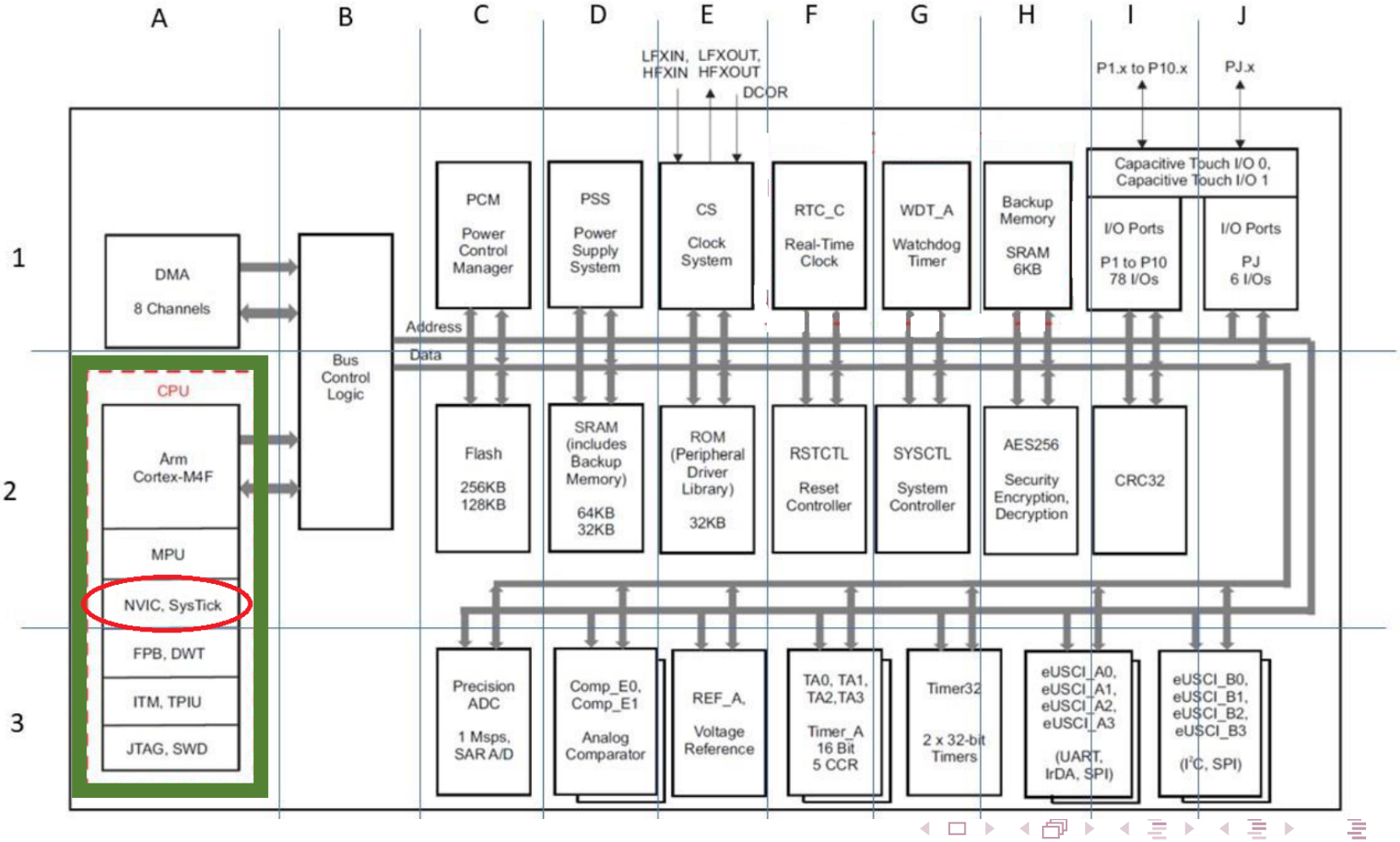
Vectored Interrupt

- For a vectored interrupt system, ISRs for different peripherals are placed at different locations in memory
- The processor must determine the correct interrupt vector for a peripheral as part of the interrupt action sequence
- This scheme is common when a microprocessor has multiple peripherals connected by a system bus
- The user must ensure that the ISR for a particular peripheral is stored starting at the memory address for that particular peripheral

Interrupts on the MSP432

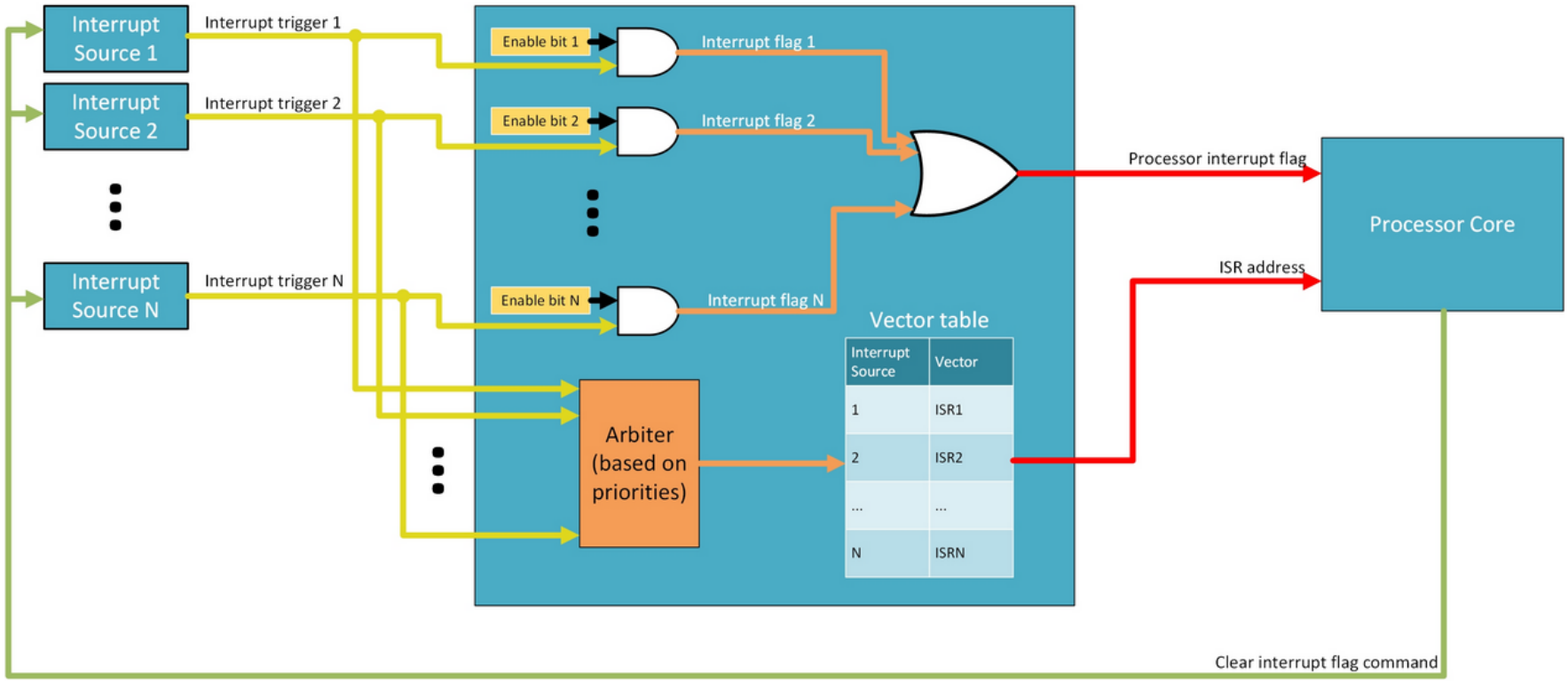
- All modern microcontrollers contain an *interrupt controller*
- On the MSP432 it is called the Nested Vectored Interrupt Controller (NVIC)

MSP432 Block Diagram



Interrupt Controller

- The programmer decides which interrupts are enabled
- The Interrupt Controller sends both the interrupt flag and the ISR address to the CPU
- Priority of interrupts decided by the arbiter



Interrupt Handlers in C

- Typically, an ISR must take some action to *clear* the interrupt request at the source
- This action is sometimes called *acknowledging* the interrupt
- If not cleared, the interrupt controller and CPU will consider the interrupt request from that source to remain active!
- Different interrupt sources may require different actions to clear them

Interrupt Handlers in C

- Typically, an ISR should execute quickly
- If the ISR takes a lot of time, the ISR effectively blocks the CPU from executing the main program
- Perform only those actions that are vital for servicing this particular interrupt
  - Good rule of thumb: an ISR should not make any function calls
- We have a complication to good programming practices:
  - The ISR cannot receive arguments or return any values
  - But an ISR can read and modify *global* variables

Interrupts – Quick Recap

- An interrupt request can arrive at any time
- The CPU can decide to handle it or not
  - Generally will handle it
  - But the CPU can decide to wait a few cycles, depending on the interrupt's priority
- CPU needs to save the context and jump to ISR
- The ISR handles the interrupt request, and then returns after restoring context

Regular Functions Versus ISR

	Regular C function	**ISR**
Can have input parameters?	Yes	No
Can return values?	Yes	No
Who decides the address of the function?	Compiler/linker/loader decide the address of the functions	It is pre-decided for each interrupt
Who decides the name of the function?	Programmer, no special registration or extra handling is need	Either a predefined name should be used or you should "register" the function so that it is known to be an ISR

Interrupts from Peripherals

- Some peripherals can automatically trigger an interrupt request, such as Timer32 when the timer expires
- Some peripherals need to be configured to cause them to generate interrupt requests, such as the GPIO
- Need to refer to data sheets and user manuals

Interrupts from Peripherals

- Interrupt trigger:
- A hardware signal from a peripheral device, requesting service
- Examples:
  - A transition on a GPIO pin
  - Timer32 expired
  - Timer_A CCR0 rollover
  - Timer_A counter becomes equal to CCRn (compare condition is met)
- Interrupt enable:
  - A bit in the Interrupt Controller that allows the interrupt trigger to pass
- Interrupt flag:
  - A bit that indicates when a valid (enabled) interrupt request has occurred
- Interrupt priorities:
  - A set of variables that decide which interrupt has priority over another
- Interrupt vector:
  - The address of the first instruction of an ISR
- Interrupt vector table:
  - An architectural entity that stores all the interrupt vectors

Summary

- Introduction to interrupts
- Polling vs. interrupt-driven operation
- Pros and cons
- Interrupt Service Routine (ISR) or the Interrupt Handler
- Programming the ISR (next lecture)
  - Harder to debug, compared to polling